Curso Avançado

Capítulo 14 - Desenvolvimento do programa completo da ferrovia

Reunir cadastro, rotas, desvios, tração, comunicação, interfaces e falhas em uma organização única de decisões.

Introdução	Os capítulos anteriores apresentaram funções separadas. Agora, vamos organizá-las em um programa integrado, mantendo uma autoridade comum para reservas e autorizações.
Objetivo	Criar uma base de integração verificável e definir as etapas para transformá-la no software da montagem real.
Meta de aprendizagem	Executar um ciclo completo de simulação, testar falhas e reconhecer quais dados e interfaces ainda precisam ser implementados na EFMN 74.

1. Integrar é definir responsabilidades

O programa precisa decidir quem interpreta eventos, quem concede recursos e quem executa comandos. Cada função fornece informações ao coordenador, e somente os pedidos permitidos seguem para a interface física.

A base deste capítulo é um sketch completo de simulação. Ele reúne uma composição, uma rota, confirmação manual de desvio, rampa de solicitação, atualização de comunicação, modos Manual/Automático e bloqueio por falha.

Ele não é o firmware definitivo da EFMN 74. Não contém as rotas oficiais, os pinos das placas, comunicação real, leitura elétrica dos sensores, atuação dos relés ou comandos de central DCC. Esses itens exigem cadastro e adaptação à montagem conferida.

2. Começar pequeno e ampliar com dados válidos

Primeiro, valide um ciclo integrado sem saídas físicas. Depois, conecte módulos e dados reais por etapas. Não copie vários sketches mantendo decisões independentes de reserva, parada ou autorização.

3. Organização do programa real

Módulo	Responsabilidade
Cadastro	Locomotivas, composições, recursos, rotas e revisões dos perfis.
Entradas	Eventos de sensores e controles, com filtragem e identificação.
Comunicação	Sessão, sequência, validade e respostas de cada executor.
Coordenador	Reserva dos recursos, etapas das rotas e autorização.
Desvios	Comando permitido e confirmação mecânica correspondente.
Tração	Solicitação limitada, rampa e interface Analógico ou DCC.
Interfaces e áudio	Apresentação e ambientação, sem conceder autoridade de circulação.
Falhas e registro	Bloqueio, aviso, preservação dos recursos e recuperação.

Cada módulo deve ter uma finalidade clara. O loop recebe informações, verifica falhas, decide etapas e atualiza tarefas. A reprodução sonora ou a atualização de uma tela não pode impedir o processamento da emergência.

4. Cadastro definitivo da EFMN 74

Confira layout vigente, rotas aprovadas, posições DI/AC, distribuição das gavetas e sensores de fronteira. Registre recursos compartilhados e locais onde toda a composição pode aguardar sem conflito.

Confirme também modelos das placas, pinos, alimentação, interfaces de tração e método de retorno das agulhas. Um número de sensor ou desvio no exemplo não define sua posição real nem uma ligação elétrica.

5. Autoridade e situação física

A reserva é uma decisão do coordenador. A ocupação é uma condição física. A confirmação de posição também depende de informação física. O firmware deve manter essas três informações distintas e reconciliá-las após falhas ou reinícios.

6. Percurso integrado do exercício

O exemplo usa L01 associada a T01 e uma rota fictícia com D6 em AC. A e B representam eventos didáticos de entrada e saída, associados a S10 e S11 somente para acompanhar o raciocínio. Não são posições reais do layout.

Etapa	Ação integrada
Inspeção	I confirma trem parado, recursos livres e cadastro conferido.
Atualização	H representa informação atual do executor.
Preparação	P reserva o percurso e apresenta solicitação de D6 em AC.
Conferência	K confirma manualmente as agulhas.
Circulação	G no Manual; rotina após K no Automático. Alvo fictício 30%.
Entrada e saída	A reduz alvo a 15%; B solicita zero e mantém reserva.
Liberação	L exige solicitação zero, parada e saída completa conferidas.
Falha	X, sequência incorreta ou prazo excedido bloqueiam e mantêm a reserva.

7. Comandos e condições de teste

M seleciona Manual e U seleciona Automático, apenas quando a operação está livre. Q mostra o estado integrado. H deve ser enviado em intervalos menores que cinco segundos enquanto houver operação reservada.

Os limites fictícios de L01 e T01 são 60% e 50%. A rampa altera um ponto percentual a cada 100 ms. Cada etapa reservada tem prazo didático de 60 segundos. Nenhum desses valores constitui um ajuste validado da ferrovia.

I é uma recuperação global simplificada: só pode ser enviado com trem parado e todos os recursos livres e conferidos. Se a composição permanecer no percurso, mantenha o bloqueio. A lógica real precisará recuperar também operações com recursos ainda ocupados.

8. Código integrado - dados, apresentação e falhas

Reúna as três partes no mesmo sketch. Configure 115200 baud. O programa não utiliza pinos nem bibliotecas de equipamentos e apresenta todos os comandos físicos como mensagens.

```
enum class Mode { MANUAL, AUTOMATIC };
enum class State { BLOCKED, IDLE, PREPARING, READY,
                   RUNNING, OCCUPIED, WAIT_CLEAR };
struct Profile {
  const char* locomotive;
  const char* consist;
  int locomotiveLimit;
  int consistLimit;
};
const Profile profile = {"L01", "T01", 60, 50}; // Fictional.
Mode mode = Mode::MANUAL;
State state = State::BLOCKED;
bool reserved = false, turnoutChecked = false, updateKnown = false;
int requestPercent = 0, targetPercent = 0;
unsigned long lastUpdate = 0, changedAt = 0, lastRamp = 0;
const unsigned long updateTimeoutMs = 5000UL;
const unsigned long stageTimeoutMs = 60000UL;
const unsigned long rampIntervalMs = 100UL;

const char* stateName() {
  switch (state) {
    case State::IDLE: return "IDLE";
    case State::PREPARING: return "PREPARING";
    case State::READY: return "READY";
    case State::RUNNING: return "RUNNING";
    case State::OCCUPIED: return "OCCUPIED";
    case State::WAIT_CLEAR: return "WAIT_CLEAR";
    default: return "BLOCKED";
  }
}
void showStatus() {
  Serial.print(profile.consist); Serial.print(" / ");
  Serial.print(profile.locomotive); Serial.print(" / ");
  Serial.println(stateName());
  Serial.println(mode == Mode::MANUAL ? "MANUAL" : "AUTOMATIC");
  Serial.print("Reserved: "); Serial.println(reserved ? "YES" : "NO");
  Serial.print("SIMULATION request / target: ");
  Serial.print(requestPercent); Serial.print(" / ");
  Serial.println(targetPercent);
}
void enterState(State next) {
  state = next;
  changedAt = millis();
}
void failOperation(const char* reason) {
  requestPercent = targetPercent = 0;
  turnoutChecked = false;
  enterState(State::BLOCKED);
  Serial.println("SIMULATION: stop and visual/audible alert.");
  Serial.println(reason);
  showStatus(); // Reservation remains held if already present.
}
```

9. Código integrado - decisões do coordenador

```
bool updateFresh() {
  return updateKnown && millis() - lastUpdate < updateTimeoutMs;
}
void setTarget(int value) {
  int limit = profile.locomotiveLimit < profile.consistLimit
    ? profile.locomotiveLimit : profile.consistLimit;
  targetPercent = value < limit ? value : limit;
  lastRamp = millis();
}
void requestRun() {
  if (state != State::READY || !reserved ||
      !turnoutChecked || !updateFresh()) {
    failOperation("Run conditions unavailable"); return;
  }
  enterState(State::RUNNING);
  setTarget(30);
  Serial.println("SIMULATION: circulation and station sound requested.");
}
void handleCommand(char c) {
  if (c == 'X') { failOperation("Emergency"); return; }
  if (c == 'Q') { showStatus(); return; }
  if (c == 'H') {
    updateKnown = true; lastUpdate = millis();
    Serial.println("Simulated fresh update. No automatic reset."); return;
  }
  if (c == 'M' || c == 'U') {
    if (state != State::IDLE || reserved) {
      Serial.println("Mode change denied."); return;
    }
    mode = c == 'M' ? Mode::MANUAL : Mode::AUTOMATIC;
    showStatus(); return;
  }
  if (c == 'I' && state == State::BLOCKED) {
    // Global manual inspection: stopped train, all resources clear.
    reserved = turnoutChecked = updateKnown = false;
    requestPercent = targetPercent = 0;
    enterState(State::IDLE);
  } else if (c == 'P' && state == State::IDLE && updateFresh()) {
    reserved = true; turnoutChecked = false;
    enterState(State::PREPARING);
    Serial.println("SIMULATION: D6 = AC. Await physical check K.");
  } else if (c == 'K' && state == State::PREPARING) {
    turnoutChecked = true; // Manual check, not a servo acknowledgement.
    enterState(State::READY);
  } else if (c == 'G' && mode == Mode::MANUAL) requestRun();
  else if (c == 'G' && mode == Mode::AUTOMATIC)
    Serial.println("Manual run request denied in AUTOMATIC.");
```

A autorização passa por requestRun nos dois modos. A informação de H é simulada e não contém confirmação de posição. K continua sendo a conferência física manual necessária ao exemplo.

10. Código integrado - eventos, rampa e supervisão

```
  else if (c == 'A' && state == State::RUNNING) {
    enterState(State::OCCUPIED);
    setTarget(15);
  } else if (c == 'B' && state == State::OCCUPIED) {
    enterState(State::WAIT_CLEAR);
    setTarget(0);
  } else if (c == 'L' && state == State::WAIT_CLEAR &&
             requestPercent == 0) {
    // Manual check: complete exit, physical stop, all resources clear.
    reserved = false;
    enterState(State::IDLE);
  } else failOperation("Unexpected command or unavailable data");
  showStatus();
}
void updateRamp() {
  if (state == State::BLOCKED || requestPercent == targetPercent) return;
  unsigned long now = millis();
  if (now - lastRamp < rampIntervalMs) return;
  lastRamp = now;
  if (requestPercent < targetPercent) ++requestPercent;
  else --requestPercent;
  Serial.print("SIMULATION traction request: ");
  Serial.println(requestPercent);
}
void supervise() {
  if (!reserved || state == State::BLOCKED) return;
  if (!updateFresh()) {
    failOperation("Executor update timeout"); return;
  }
  if (millis() - changedAt >= stageTimeoutMs)
    failOperation("Operation stage timeout");
}
void setup() {
  Serial.begin(115200);
  failOperation("Startup: inspect before I.");
}
void loop() {
  supervise();
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c != '\n' && c != '\r' && c != ' ') handleCommand(c);
  }
  if (mode == Mode::AUTOMATIC && state == State::READY) requestRun();
  updateRamp();
}
```

11. Limites desta base de integração

O código não identifica locomotivas automaticamente nem detecta a ocupação contínua. A composição e os perfis são constantes fictícias. A reserva representa um único recurso agregado, sem arbitrar vários trens.

Comunicação, áudio e tela são apenas mensagens. O programa também não mantém um registro persistente ou a primeira causa de falha. Para a montagem definitiva, implemente esses módulos conforme as regras dos capítulos anteriores e a situação física conferida.

12. Roteiro de validação integrado

Teste	Resultado esperado
I H P K G no Manual	Preparação conferida e autorização simulada.
A; depois B; aguardar zero; L	Redução, parada solicitada e liberação após conferência completa.
I U H P K; depois A e B	Automático solicita passagem após K. Sem repetir ciclo sozinho.
P sem atualização H válida	Bloqueio por informação indisponível.
G antes de K	Bloqueio e manutenção da reserva.
M ou U durante reserva	Troca recusada, modo vigente mantido.
Parar H por mais de 5 s	Falha de atualização e reserva preservada.
X em qualquer etapa	Solicitação zero imediata e bloqueio.
L antes de solicitação zero	Bloqueio. Liberação antecipada recusada.

Envie H regularmente durante o ciclo. Separe os eventos nos momentos previstos. Em Automático, aguarde a mensagem de autorização após K antes de enviar A. O período da rampa não confirma a velocidade ou a parada física.

13. Passagem da simulação para a montagem

Integre primeiro uma leitura real, verificando nível contínuo e evento filtrado. Depois, conecte um desvio e confira o retorno de posição. Mantenha a tração desativada até conferir os comandos e a proteção física.

Teste a comunicação entre controle e uma gaveta, incluindo perda de atualização e reinício. Acrescente a interface de tração somente com o percurso e a composição definidos. Amplie para outras rotas e gavetas após validar conflitos, reservas e recuperação.

Cada etapa deve ter resultado esperado e condição de bloqueio. A revisão dos testes precisa acompanhar qualquer alteração do layout, do cadastro ou dos equipamentos.

14. Exercício e registro no Diário de Bordo

Monte uma lista dos módulos definitivos, dados pendentes e interfaces necessárias. Registre os testes integrados, a versão do código e a montagem utilizada. Identifique claramente o que já foi validado e o que continua somente em simulação.